



CAPSTONE JUNTA DE VECINOS UT

Integrantes: Miguel Medina, William
López

PROBLEMA

Actualmente las juntas de vecinos enfrentan procesos lentos, poco claros y sin herramientas digitales que respalden su trabajo. Los vecinos no cuentan con un canal eficiente de comunicación, los trámites son manuales y el uso de salones o canchas de la junta de vecinos no tiene orden.

- Gestión manual y fragmentada
- Falta de comunicación efectiva
- Escasa transparencia y trazabilidad
- Acceso limitado a recursos comunitarios
- Desorganización en el uso de recursos
- No existe canal digital de incidencias

SOLUCIÓN

La solución es un sistema web responsivo que digitaliza la gestión de la junta, permitiendo automatizar trámites, mejorar la comunicación, garantizar transparencia en los procesos y facilitar la inclusión digital de todos los vecinos.

- Plataforma web responsiva (Django + Supabase(PostgreSQL))
- Inscripción y padrón de vecinos
- Certificados con PDF + firma digital
- Noticias y avisos comunitarios
- Reservas de recursos (anti-solape)
- Incidencias con fotos
- Notificaciones por email

BENEFICIOS

El sistema permite reducir los tiempos de atención, centralizar la comunicación con los vecinos, dar trazabilidad a cada trámite, ordenar las reservas de recursos comunitarios.

- 50% menos tiempo en trámites
- Comunicación clara y unificada

- Transparencia en solicitudes
- Uso ordenado de recursos

- Canal formal de incidencias

METODOLOGIA

El proyecto se gestiona con la metodología iterativa incremental, dividiendo el trabajo en dos personas mejorándolo en 7 iteraciones. Se establecieron hitos claros que marcan la entrega de un MVP, una versión beta y el producto final.

- 0 a 6 iteraciones
- Hitos principales:
- Planificación de iteraciones
- Desarrollo y pruebas
- Revisión y ajustes

ARQUITECTURA Y DISEÑO

El diseño del sistema se basa en el modelo-vista-controlador (MVC) y un ERD que representan las entidades clave y sus relaciones. Esto asegura consistencia de datos y trazabilidad de procesos, garantizando que cada caso de uso tenga respaldo en la estructura técnica.

- Diagrama de Clases (resumido)
- ERD simplificado
- Casos de Uso

CRONOGRAMA

- El plan de ejecución contempla 6 iteraciones más la fase inicial de preparación, que permiten entregar valor de manera incremental, comenzando con funcionalidades críticas como la autenticación y el padrón, hasta llegar a la entrega final con todas las características del sistema:
- Iteración 0 (Semana 1): Incepción y preparación
- Iteración 1 (Semanas 2-3): Autenticación + padrón
- Iteración 2 (Semanas 4-5): Documentos + avisos
- Iteración 3 (Semanas 6-7): Correos + documentos
- Iteración 4 (Semanas 8-9): Reservas (anti-solape)
- Iteración 5 (Semanas 10-11): Incidencias
- Iteración 6 (Semana 12): Cierre y documentación

COSTOS

El proyecto se diseñó con un enfoque low-cost, aprovechando herramientas open source y servicios en la nube con planes gratuitos o de bajo costo.

- Infraestructura: \$0-20 USD/mes
- Herramientas de apoyo: \$0 - \$15 USD/mes
- Principal costo: tiempo de desarrollo (600-620 horas)
- Valor referencial: \$2.880.000 - \$3.160.000 CLP (mercado real)

MÉTRICAS DE PRUEBA

Métricas Funcionales:

- Casos de pruebas ejecutados: 33
- Casos de prueba funcionando correctamente: 33
- Número de módulos completamente operativos: 7

Métricas de Rendimiento:

- Tiempo promedio de carga de vistas: 2-3s
- Tiempo para enviar un certificado: 5s
- Tiempo para solicitar reserva: 2
- Validación de un pago: 4s

Métricas de Seguridad:

- Recuperación de contraseña funcionando
- Intentos inválidos rechazados: 100%
- Accesos no autorizados permitidos: 0

LECCIONES APRENDIDAS

- La planificación inicial siempre cambia: el sistema evolucionó respecto al plan original, permitiendo mejoras técnicas y funcionales.
- La metodología iterativa facilitó el avance: entregar versiones frecuentes permitió corregir rápido y reducir riesgos.
 - Centralizar la base de datos en Supabase fue clave: simplificó el trabajo colaborativo y redujo problemas de sincronización
- El proyecto creció más de lo esperado: fue necesario priorizar funcionalidades y ajustar el alcance durante el desarrollo.